

Assignment 1: Fine-Tune, Sanitize, Extract

ELL8299/COL8383/COL873: Trustworthy Large Language Models - Semester I, 2026-27
Release Date: 31/08/2026 | Due By: 14/09/2026, 05:00 PM IST | Marks: 75

Objective

Fine-tuning LLMs on unvetted data often leads to the unintended inclusion of private information in the model's weights. In this assignment, you will implement an end-to-end pipeline to analyze and mitigate this risk:

1. **Efficient Fine-tuning:** Utilize PEFT (LoRA, QLoRA) to adapt a small instruct model to serve as a PII sanitizer. Evaluate adapter placement and hyperparameters.
2. **Sanitization Assessment:** Evaluate sanitizer efficacy using privacy-centric metrics.
3. **Extraction Attacks:** Fine-tune a "victim" model on a PII-rich corpus containing planted canaries. Execute extraction attacks, including discoverable memorization, membership scoring, and targeted PII prompting.
4. **Analysis:** Subject a model trained on sanitized data to the same attacks. Quantify the privacy-utility trade-off by measuring the reduction in leakage relative to the loss in model performance.

This assignment consists of three parts: efficient fine-tuning of a PII sanitizer using PEFT, rigorous evaluation of its sanitization performance, and training-data extraction attacks to study memorization and the privacy-utility trade-off.

Provided Dataset Bundle ([click here to download](#))

All data needed for the assignment is provided in the `data/` directory of the .zip file in .jsonl format. Do not download, resample, replace, or augment these files. You must use the supplied rows in the given order.

File	Rows	Purpose
<code>partA_train.jsonl</code>	20,000	English sanitizer training split
<code>partA_validation.jsonl</code>	2,000	English model-selection split
<code>partB_test.jsonl</code>	2,000	Held-out English in-distribution test split
<code>partB_ood_test.jsonl</code>	1,000	OOD test split
<code>partC_corpus_raw.jsonl</code>	8,007	Expanded raw victim-training corpus (5,000 distinct base documents plus controlled repeats and canaries)
<code>partC_heldout.jsonl</code>	500	Never-trained utility and attack-calibration documents
<code>partC_canaries.jsonl</code>	20	Canary secrets, templates, and repetition metadata
<code>partC_targeted.jsonl</code>	400	200 member and 200 non-member control prompts with gold values

Part A/B records have the following schema:

```
{"id": "a_train...", "language": "English", "source_text": "...", "target_text": "...", "privacy_mask": [{"start": 0, "end": 4, "label": "...", "value": "..."}], "split": "train"}
```

Part C corpus and held-out records have the following fields: {"doc_id", "text", "n_reps", "is_canary", "canary_id", "repetition_index"}.

Repeated rows share a `doc_id`; `repetition_index` runs from 0 through `n_reps - 1`. The canary and targeted-attack schemas are described in Part C.

Part A: PEFT: Where Do You Put the Adapters? (25 marks)

Task. Fine-tune Qwen2.5-1.5B-Instruct to map `source_text` to `target_text` using `data/partA_train.jsonl`; use `data/partA_validation.jsonl` for model selection. This is a supervised instruction-tuning task: a fixed system/user prompt containing the raw text, and a target completion containing the masked text. Compute loss on completion tokens only (mask the prompt). Which axis (placement/rank/ α) bought the most F1 per unit of trainable parameters, per unit of memory, and per unit of wall-clock time? Where does the curve flatten? Did QLoRA cost you accuracy, and how much memory did it save?

Run at least these six configurations, all with the same data, seed, epochs, LR schedule, and batch size:

#	Method	target_modules	r	α
A1	LoRA	q_proj, v_proj	8	16
A2	LoRA	q_proj, k_proj, v_proj, o_proj	8	16
A3	LoRA	attention + gate_proj, up_proj, down_proj	8	16
A4	LoRA	same as A3	32	64
A5	LoRA	same as A3	32	16
A6	QLoRA	same as A3	32	64

Evaluate all configurations using the metrics in Part B. Use `model.print_trainable_parameters()` and `torch.cuda.max_memory_allocated()`. Do not estimate these by hand. Use all and only the supplied 20,000 training and 2,000 validation examples for ≤ 10 epochs. Do not resplit or subsample. Reuse the identical files across all runs.

Part B: How Good Is Your Sanitizer, Really? (25 marks)

Take your best Part A checkpoint (justify "best", it need not be the highest F1) and evaluate it as a sanitization system on `data/partB_test.jsonl`.

B.1 Span recovery. Your model emits masked text, not spans. Recover predicted spans by aligning `source_text` against the generated output (e.g., `difflib.SequenceMatcher`), so that each replaced region becomes a (start, end, predicted_label) triple. Document your alignment rules and their failure modes.

B.2 Metrics. Against the ground truth `privacy_mask`, all of the following have to be implemented:

- Entity-level precision/recall/F1, micro and macro over PII classes, under two matching criteria:
 - exact span match + label match,
 - partial overlap ($\text{IoU} \geq 0.5$) + label match.Report a per-class table for the 10 most frequent classes.
- Leak rate: the fraction of gold PII values that still appear verbatim (case-insensitive, whitespace-normalized) in the sanitized output.
- Over-masking rate: the fraction of non-PII characters that were removed or replaced.

B.3 Baselines. Compare against at least two non-finetuned baselines on the same test set and report all metrics for every baseline in one table:

- a regex/rule baseline you write yourself (email, phone, IP, credit card, date, URL, etc.);

- [Presidio](#) with its default recognizers, or the same Qwen2.5-1.5B-Instruct model with zero-shot/5-shot settings (no fine-tuning).

B.4 Generalization. Evaluate your sanitizer on the supplied `data/partB_ood_test.jsonl`. Report the same metrics and explain the difference. We will run your code on a private test set and a small OOD subset.

Note: For both exact and IoU matching, perform one-to-one matching between predicted and gold entities. A predicted entity and a gold entity can each participate in at most one match. For IoU matching, among valid same-label pairs with $\text{IoU} \geq 0.5$, use maximum-IoU matching.

Part C: Training Data Extraction Attacks (25 marks)

The goal of this part is to determine whether information present in the victim-model training corpus can be recovered from the trained model, and whether sanitizing the training corpus reduces such leakage. You will compare three models throughout Part C:

- V-base: the unmodified Qwen/Qwen2.5-0.5B base language model. It is never fine-tuned on the Part C corpus.
- V-raw: Qwen/Qwen2.5-0.5B fine-tuned on the supplied raw corpus.
- V-san: the same base model fine-tuned on a sanitized version of exactly the same corpus.

The victim models must be base causal language models, not instruction-tuned models. V-raw and V-san must use identical training hyperparameters and PEFT configurations. The only intended difference between them is the text used for training.

Supplied Leaky Corpus: The supplied raw corpus contains 5,000 distinct base documents, controlled repetitions, and planted canaries. Repeated rows share the same `doc_id`; `repetition_index` ranges from 0 to `n_reps - 1`. The held-out corpus contains 500 distinct documents that do not appear in the training corpus. Use it for perplexity/utility comparison and attack calibration. The canaries file contains one row per canary with fields `{"canary_id", "template", "project", "secret", "n_reps", "seed"}`. Treat this file as evaluation metadata; i.e., never train on it separately or expose it to the model in a prompt, except as required by the canary-ranking attack.

C.1 Victim Model Training: Train two models using identical hyperparameters and PEFT configurations:

- V-raw: fine-tuned on `data/partC_corpus_raw.jsonl`.
- V-san: first create `data/partC_corpus_sanitized.jsonl` locally by passing the `text` of every raw row through your Part B sanitizer while preserving all metadata fields, then fine-tune on that file. Use the same deterministic sanitizer-decoding configuration as in your Part B evaluation. **Do not special-case or manually remove canaries.** If the sanitizer fails to remove a planted secret, that is an observed sanitizer failure and must remain in V-san's training corpus.

Evaluate held-out perplexity for both models against the pre-trained (base model) baseline. For the entire held-out corpus, compute a token-weighted perplexity; do not compute the average of document-level perplexities.

C.2 Attack Implementation: Execute four attacks against both victims and the control model:

(a) Discoverable Memorization: Assess greedy decoding accuracy on training prefixes, report exact-match rates by repetition count.

(b) Untargeted Extraction: Generate 5,000 samples and rank via membership scoring (Appendix A). Report `precision@100` and PII recovery rates.

(c) Canary Exposure: Estimate exposure levels by ranking true secrets against 9,999 alternatives.

$$\text{exposure} \approx \log_2(|R|) - \log_2(\widehat{\text{rank}}) \text{ where } \widehat{\text{rank}} = \left(\frac{\text{rank}_{\text{sample}}}{10000}\right)|R|, \quad |R| = 10^6$$

Plot mean exposure vs. repetition count for V-raw and V-san on the same axes. State the limitation of the sampled-rank estimator at high exposure.

(d) Targeted PII Extraction: Use `data/partC_targeted.jsonl`, whose schema is `{"id", "split", "doc_id", "prompt", "gold_value", "label"}`. It contains 200 member prompts and 200 held-out controls.

Prompt each model with `prompt` and test continuation recovery of `gold_value`; report greedy and sampled leak rates separately for members and controls.

C.3 Comparative Analysis: Quantify sanitization efficacy vs. utility cost. Identify specific sanitizer failures that permitted leakage.

Ethics and Ground Rules

These attacks are taught for defensive research purposes. Adherence to the following ethical mandates is a requirement of academic integrity:

- **Scope:** Only attack models fine-tuned within this assignment. Attacking third-party or production services (e.g., OpenAI, Anthropic, or institutional systems) is strictly prohibited.
- **Data Privacy:** Use only the provided synthetic (ai4privacy) or public (Enron) datasets. Do not scrape or utilize real personal data belonging to individuals (e.g., classmates, faculty, or social media users).
- **Reporting Standards:** Redact any extracted PII in your PDF report (e.g., user***@enron.com). Present results as aggregate counts and rates; do not include raw data dumps in the final document.
- **Distribution:** Do not upload fine-tuned victim models or extraction outputs to public repositories such as HuggingFace Hub or GitHub.

Model Specifications

The base/instruct distinction must be maintained; victim models must be base LMs to ensure the validity of causal-LM fine-tuning. Models previously fine-tuned on PII-masking data are strictly prohibited.

Deliverables and Submission

Submit a ZIP file ([EntryNumber].zip, <= 500MB) containing:

- **Documentation:** Report (≤5 pages), requirements.txt, run_all.sh, and README.md.
- **Source Code:** GitHub repository or ZIP file. Share GitHub repository access with the TAs using their usernames: sudiptog81, vinay-kumar07, proadhikary, tkarhikeyan132.
- **Report (PDF) with Results, Plots, and Adapters** (provide a link accessible to TAs)

Follow this directory structure for the source code:

```
├── README.md
├── src/
│   ├── peft_sweep.py
│   ├── sanitize_eval.py
│   ├── train_victim.py
│   └── attack_extract.py
└── results/
    ├── partA_sweep.csv
    ├── partA_fig.png|pdf
    ├── partB_metrics.json
    ├── partB_predictions.jsonl
    ├── partC_attacks.json
    └── partC_exposure.png|pdf
```

You can have additional files; however, all of the above must be present. Do not submit any base models, full corpus text, generation dumps, or any wandb/run directories. Total ZIP size should be <= 500MB.

Required CLI interfaces

We will invoke these directly. Extra arguments are fine; these must exist and behave as specified.

```
python src/peft_sweep.py \
```

```

--model Qwen/Qwen2.5-1.5B-Instruct \
--train_file data/partA_train.jsonl --val_file data/partA_validation.jsonl \
--target_modules q_proj,k_proj,v_proj,o_proj,gate_proj,up_proj,down_proj \
--lora_r 32 --lora_alpha 64 --lora_dropout 0.05 \
--quant {none,4bit} --seed 42 \
--run_id A4 --output_dir adapters/A4 \
--results_csv results/partA_sweep.csv # appends one row to results_csv

python src/sanitize_eval.py \
--base_model Qwen/Qwen2.5-1.5B-Instruct \
--adapter_path adapters/best_sanitizer \
--test_file data/partB_test.jsonl \
--system {finetuned,regex,presidio,zeroshot} \
--output_predictions results/partB_predictions.jsonl \
--report_file results/partB_metrics.json

python src/train_victim.py \
--model Qwen/Qwen2.5-0.5B --corpus data/partC_corpus_raw.jsonl \
--heldout data/partC_heldout.jsonl --target_modules ... \
--lora_r 32 --lora_alpha 64 --epochs 4 --seed 42 \
--output_dir adapters/victim_raw

python src/attack_extract.py \
--victim_model Qwen/Qwen2.5-0.5B \
--adapter_path adapters/victim_raw \
--ref_model Qwen/Qwen2.5-0.5B \
--corpus data/partC_corpus_raw.jsonl \
--heldout data/partC_heldout.jsonl \
--canaries data/partC_canaries.jsonl \
--pii_index data/partC_pii_index.jsonl \
--targeted_file data/partC_targeted.jsonl \
--attack {discoverable,untargeted,canary,targeted,all} \
--num_samples 5000 \
--seed 42 \
--report_file results/partC_attacks.json

```

File Format Specifications

results/partA_sweep.csv:

These columns should be there.

```

run_id,method,target_modules,lora_r,lora_alpha,quant,trainable_params,total_params,trainable_pct,peak_gpu_mem_gb,train_wallclock_s,tokens_per_s,final_train_loss,
val_loss,val_entity_f1,val_leak_rate

```

results/partA_fig.(png|pdf):

val_entity_f1 vs trainable_pct (log scale), points annotated with run id.

(Another panel in the same figure with a plot of peak_gpu_mem_gb vs. val_entity_f1 is also encouraged.)

results/partB_predictions.jsonl (one line per test example):

```

{
  "id": "en_00412",
  "language": "en",
  "source_text": "Please contact Marta Ruiz at m.ruiz@example.org before 5pm.",
  "predicted_text": "Please contact [GIVENNAME_1] [SURNAME_1] at [EMAIL_1] before [TIME_1].",
}

```

```

    "predicted_spans": [
      {
        "start": 15,
        "end": 20,
        "label": "GIVENNAME"
      },
      {
        "start": 21,
        "end": 25,
        "label": "SURNAME"
      }
    ],
    "gold_spans": [
      {
        "start": 15,
        "end": 25,
        "label": "PERSON"
      }
    ],
    "leaked_values": [],
    "latency_ms": 184
  }

```

results/partB_metrics.json:

```

{
  "system": "finetuned",
  "adapter": "adapters/best_sanitizer",
  "test_set": "ai4privacy_val_en",
  "n": 2000,
  "exact": {
    "micro_p": 0,
    "micro_r": 0,
    "micro_f1": 0,
    "macro_f1": 0
  },
  "overlap_iou50": {
    "micro_p": 0,
    "micro_r": 0,
    "micro_f1": 0,
    "macro_f1": 0
  },
  "per_class": {
    "EMAIL": {
      "p": 0,
      "r": 0,
      "f1": 0,
      "support": 0
    }
  },
  "leak_rate": 0,
  "over_masking_rate": 0,
  "well_formed_rate": 0
}

```

results/partC_attacks.json:

```

{
  "victim": "victim_raw",
  "heldout_ppl": 0.0,

```

```

"discoverable": {
  "n_documents": 0,
  "n_excluded_short": 0,
  "exact_match_rate": 0.0,
  "mean_edit_sim": 0.0,
  "by_n_reps": {
    "1": {
      "n": 0,
      "exact_match_rate": 0.0
    }
  }
},

"untargeted": {
  "n_generated": 5000,
  "n_unique": 0,

  "precision_at_100": {
    "ppl": 0.0,
    "ratio_ref": 0.0,
    "ratio_lower": 0.0,
    "ratio_zlib": 0.0
  },

  "distinct_docs_recovered": 0,
  "distinct_pii_values_recovered": 0,
  "pii_recovery_rate": 0.0
},

"canary": {
  "n_canaries": 20,
  "mean_exposure": 0.0,
  "by_n_reps": {
    "1": {
      "n": 0,
      "mean_exposure": 0.0
    }
  }
},

"targeted": {
  "n_member": 200,
  "n_control": 200,

  "member": {
    "greedy_leak_rate": 0.0,
    "sampled_any10_leak_rate": 0.0
  },

  "control": {
    "greedy_leak_rate": 0.0,
    "sampled_any10_leak_rate": 0.0
  }
}
}

```

Extra keys are permitted. Every number must be reproducible from your `run_all.sh` with the seeds you report.

Report

You are expected to submit a report (PDF) containing no more than 5 pages. The following must be included in the report.

1. Part A: sweep table, figure, and the placement/rank/ α analysis.
2. Part B: metric table across systems and baselines, OOD results, and an error taxonomy of your sanitizer's misses (with counts).
3. Part C: the base/V-raw/V-san comparison table, exposure vs. repetitions plot, and the two traced leaks.
4. What would you do next? Given the leakage you measured, what defense would you build?

We will look for (i) the required result or comparison, (ii) enough experimental detail to interpret it, (iii) conclusions supported by the submitted numbers, and (iv) clear acknowledgment of limitations or failures. Merely restating the task or including a table without analysis does not earn analysis marks. Report marks are awarded independently in Parts A, B, and C.

An optional appendix may contain extra tables, prompts, and hyperparameters. Also, include the honour code verbatim: "Even though I have taken help from the following students and LLMs in terms of discussing ideas and coding practices, all my code is written by me." Also, list the names of every student you consulted/discussed the problems with, and every LLM/tool you used, or write "None".

Computational Constraints and Limits

- **Reference Environment:** Python 3.10, T4/P100/V100 GPU.
- **Library Policy:** torch, transformers, peft, trl, datasets, matplotlib, pandas, numpy, seaborn, vllm, tqdm, presidio-analyzer, presidio-anonymizer, bitsandbytes, accelerate, and their dependencies. Other requests must be approved via Piazza.
- **Execution Time Limits:** Held-out evaluation is capped at 90 minutes for inference runs and 180 minutes for extraction attacks.
- **vLLM policy:** vLLM is permitted for inference and likelihood scoring in Part A validation, Part B, and Part C evaluation/attacks. You should disable model-repository generation defaults or explicitly set every sampling parameter. vLLM can otherwise apply a model's generation_config.json. Trainable parameter counts, GPU memory usage, throughput, and wall-clock measurements must be collected from the training process.

Academic Integrity and Late Policy

Fabricated data results in a zero grade for the associated component. Collaborative conceptual discussion is encouraged; however, all code must be original. LLM usage is restricted to boilerplate/debugging and must be disclosed. Plagiarism detection will be strictly enforced. Late submissions will not be accepted.

Clarifications

Post all questions to the Piazza forum. It is your responsibility to seek clarification before the deadline.

Appendix A: Membership Scores

Let $L_V(x)$ denote the mean token NLL of the generated sequence x under the tested victim model.

1. PPL score:

$$s_{\text{ppl}}(x) = \exp(L_V(x)).$$

2. Reference-model ratio:

Let V_{base} denote the untouched Qwen2.5-0.5B model.

Define

$$s_{\text{ref}}(x) = \frac{\text{PPL}_V(x)}{\text{PPL}_{V_{\text{base}}}(x)} = \exp[L_V(x) - L_{V_{\text{base}}}(x)].$$

A particularly small value means that the victim model finds the sequence much more likely than the untouched reference model does.

3. Lowercase ratio

Let $\text{lower}(x)$ be the Unicode-lowercased version of x .

Define

$$s_{\text{lower}}(x) = \frac{\text{PPL}_V(x)}{\text{PPL}_V(\text{lower}(x))}.$$

This score attempts to discount sequences whose apparent likelihood is largely explained by ordinary lexical/statistical structure.

4. Zlib-normalized score

Let

$$H_{\text{zlib}}(x) = \frac{8 |\text{zlib}(\text{UTF8}(x))|}{|\text{UTF8}(x)|}$$

be the compressed number of bits per original byte. Define

$$s_{\text{zlib}}(x) = \frac{L_V(x)}{H_{\text{zlib}}(x)}.$$

Use Python's standard `zlib.compress` with its default compression level for all submissions.